

Stack-Overflow小组大作业报告：睡眠管理系统

一、程序功能介绍

我们制作了用户说明书，详情请见说明书（说明书在程序打开时会自动弹出，也可参考github中的README文件）
本项目是一款面向大学生群体的健康作息管理系统，核心目标是解决大学生“睡得晚、起得乱、记录难、复盘少”的问题。程序基于 Qt 图形界面开发，通过日历、打卡按钮、系统托盘、桌面悬浮球、图表窗口、设置窗口等交互方式，帮助用户完成每日作息记录、睡眠评分、历史回顾、周报分析和健康提醒。

系统主要功能包括：

- 作息数据记录**：记录入睡时间、起床时间、午睡时长、运动时长、久坐时长和睡眠日记。
- 作息日逻辑**：将某一天白天活动与当天夜间开始、次日早上结束的睡眠绑定为同一个“作息日”，避免跨天睡眠导致数据归属混乱。
- 手动补录与修正**：支持手动修改入睡/起床时间，支持补录过去日期，支持“我熬穿了”按钮标记通宵。
- 本地 JSON 持久化**：每个作息日以一个 `yyyy-MM-dd.json` 文件保存，实现历史记录读取、回填和删除。
- 睡眠评分与每日简报**：根据夜间睡眠时长、是否通宵、是否睡懒觉等因素计算评分，并生成趣味化日报报告。
- 日历颜色标记**：在日历中用不同颜色标记通宵、懒觉、一般、正常、良好等睡眠状态。
- 成就系统**：根据连续打卡、连续早睡、连续良好睡眠等条件解锁徽章。
- 近七天周报**：无 API Key 时使用本地算法生成周报；填写 DeepSeek API Key 后可调用大模型生成“修仙宗门掌律大长老”风格周报。
- PDF 导出**：将周报内容导出为 PDF 文件。
- 睡眠图表**：绘制近七天睡眠时长柱状图和睡眠评分折线图。
- 键盘注意力曲线**：跨平台统计每分钟键盘敲击次数，并绘制键盘活跃度图表。
- 系统托盘与桌面提醒**：关闭窗口后最小化到托盘，后台继续守护，并在深夜未睡时弹出提醒。
- 桌面悬浮球**：提供主窗口外的一键入睡、一键起床快捷入口。
- 用户自定义设置**：用户可设置一般入睡时间、一般起床时间和熬夜判定区间。
- 用户说明书**：程序启动时自动弹出说明书，也可通过帮助按钮再次查看。

我们制作了用户说明书，详情可见程序内置说明书；GitHub 中的 README 文件也对功能、结构、运行方式和数据格式进行了完整说明。

二、项目各模块与类设计细节

项目整体架构

本项目整体采用“核心算法层 + Qt 界面层 + 本地持久化层 + 系统能力适配层”的结构。

- 核心算法层**主要集中在 `sleep_core.h`，负责睡眠数据建模、作息日判断、评分、周报统计与 JSON 导出，不直接依赖界面。
- 界面交互层**主要由 `mainwindow.ui`、`mainwindow.h`、`mainwindow.cpp` 组成，负责用户输入、按钮响应、日历回填、弹窗、图表调用、AI 请求和 PDF 导出。

- **本地持久化层**通过 Qt 的 `QFile`、`QJsonDocument`、`QStandardPaths` 等类实现，把用户数据和配置保存到系统应用数据目录。
- **系统能力适配层**包括系统托盘、桌面通知、键盘监测、平台差异编译、资源打包和自动构建等内容。

这种划分使得睡眠算法、界面显示、系统服务和辅助工具之间相对独立，便于后期维护和扩展。

sleep_core.h

这个文件只负责“纯数据和算法”，主要部分不依赖具体 UI。负责基本的C++可实现的全局变量定义、作息日数据分析、周报分析与JSON文件生成。

基类：SleepData

作为每日数据的基本存储单位，是数据分析类的基类

单日数据分析派生类：SleepAnalyzer

基于基类 `SleepData` 提供的数据所构建，负责自动统计每日睡眠时长、一周规律及熬夜天数，计算当日睡眠评分，包含多个与计算与输出显示相关的功能性函数，例如：

- `calculateNightSleep()`：计算晚上睡眠时间，并判断是否有晚起床（晚于平时两小时）现象
- `isStayUpLate`：判断是否熬夜，如果在设定的熬夜区间内入睡就算作熬夜
- `getEnoughSleepScore`：根据夜晚睡眠时长计算睡眠分数；区分了熬穿、睡懒觉（起太晚）、正常睡眠、睡太长这些情况

周数据分析类：WeeklyTracker

将一整周每天的SleepAnalyzer对象作为核心变量，进行分析并综合评分，最后根据分数提供本地算法周报。

JSON文件生成

`SleepJsonExporter` 这个类负责创建JSON格式的数据文件的所有操作。

通过 `ostringstream` 把所有数据拼成JSON格式的字符串，并进行转义。

mainwindow.h & mainwindow.cpp

负责定义主窗口类的变量及函数，以及具体的实现。包含所有的信号-槽函数（private slots）以及与信号无关的辅助函数（private）。public部分只包含构造函数、析构函数与退出窗口需要实现的函数。

具体函数的定义可查看代码注释。

achievement_manager.h

成就管理器。负责成就的判定和显示逻辑，具体函数在mainwindow.cpp内调用。

行为状态计算

通过 `consecutiveCheckIn()`、`consecutiveEarlySleep()`、`perfectSleepDays()`、`goodSleepDays()` 四个函数计算四种成就，并在调用时满足只在完全等于指定日期的那一天才会触发。

JSON 数据解析与时间回滚算法

在私有辅助函数 `readRecord()` 中，利用 `QFile` 打开对应日期的文本，通过 `QJsonDocument::fromJson` 将其反序列化为 `QJsonObject`，提取出 "sleep_hour" 等数据。

数据读取逻辑从今天 (`QDate::currentDate()`) 开始作为基准点，利用循环配合 `.addDays(-i)` 逐天向过去回溯。考虑到了调用函数的当天可能破坏判断逻辑，因此从昨天开始读取数据，只读取三个月内满足条件的最近的时间段。

非侵入式纯逻辑设计

不继承任何 Qt 的 UI 类，内部也没有任何界面弹窗代码。它只接收数据目录路径 `m_dir`，通过暴露轻量级的公共接口（如 `consecutiveCheckIn()`、`currentBadge()`）向外（`mainwindow.cpp`）输出分析结果。

settings_dialog.h & settings_dialog.cpp

管理用户自定义设置的配置文件，涉及了磁盘文件的写入读出，并负责提供用户自定义参数的界面控件操作逻辑。

本地配置的持久化存储引擎 (I/O 模块)

提供了 4 个面向底层磁盘 I/O 的结构化 C 风格全局函数，负责将配置信息以加密/结构化的 JSON 文本持久化保存在本地：

- `defaultUserConfig()`：生成一套默认配置（默认入睡23点、起床8点、0-8点算熬夜），作为系统初次运行或配置丢失时的底线。
- `loadUserConfig(path)`：使用 `QFile` 和 `QJsonDocument` 读取本地文件，并在文件不存在或损坏时具有**防崩溃的自动兜底机制**。
- `saveUserConfig(path, config)`：将内存中的 JSON 对象序列化转换为 UTF-8 文本流，安全写入磁盘。
- `applyUserConfig(config)`：将读取出来的用户参数动态赋值到 `sleep_core.h` 的全局变量中，实现配置对全局算分引擎的即时生效。

数据驱动的架构

`SettingsDialog` 所控制的窗口，并不直接操作零散的 UI 控件数据，而是采用 `QJsonObject` (JSON对象) 作为统一的数据载体。

- **构造函数**：接收一个包含当前配置的 `QJsonObject` 并自动初始化界面控件。
- **输出接口**：提供 `getConfig() const` 公开函数，将用户在界面上所有修改后的结果，再次打包成一个 `QJsonObject` 返回给主窗口。

这种设计使得界面层设计与底层逻辑解耦，主窗口和配置窗口之间只通过一个标准的 JSON 对象进行数据交互。

数据挂载机制

为了用最简单的方式将原本只有整点可选择的时间，修改为可选择半点的时间，使用了 `QComboBox`（下拉单选框）让用户选择时间。数据挂载机制指的是前端显示的是格式化的字符串如 "23:00"，但后台实际记录与操作的是浮点数 23.0，简化了“时间字符串”与“数学逻辑分析”之间的转换。

多级信号和槽的联动

在防止用户设置出不符合逻辑的作息时间表（例如：设置“23:00入睡，22:00起床”这种逆向时间），设计了信号槽间（控件间）的**交互约束**，也算一种防呆设计吧。

用户在“入睡时间”下拉框中选择了某个时间（如 23:00），`onGeneralSleepChanged()` 会被触发，它会**动态重新计算并刷新**“起床时间”下拉框的可选范围（比如限制只能选择入睡后 1h 至 16h 之间的时间），并处理越界纠错逻辑。

sleep_charts.h & sleep_charts.cpp

负责绘制近七天睡眠数据的柱状图和折线图，使用了高级自定义绘制（Custom Paint）技术，根据睡眠时长给予不同的颜色，提供睡眠时长趋势的可视化分析。

基于QPainter 的2D 绘图引擎

通过重写 `paintEvent(QPaintEvent*)` 虚函数，利用 Qt 的 `QPainter` 机制进行硬编码绘制。

- `drawBackground`：绘制底色与图表画布区域。
- `drawGrid`：计算最大值和步长，自适应绘制灰色参考网格线（`COLOR_GRID`）。
- `drawLabels`：利用 `QFontMetrics` 动态计算文本的像素宽度，渲染 X 轴日期和 Y 轴刻度文本。
- `drawBars` / `drawLineChart`：绘制图形（柱状图或折线图）。

低耦合的组件组合模式

我们没有将所有图表混在一起写，而是拆分成了两个独立的、继承自 `QWidget` 的图形专用控件：`SleepBarChart`（柱状图组件）和 `SleepScoreChart`（折线趋势图组件）。而最后，通过一个 `WeekChartDialog`（对话框组件），利用 `QVBoxLayout` 将这两个控件组合在一起。

对于柱状图，图表内嵌了颜色状态常量，并采用了双层堆叠柱状图，浅紫色的午睡时长堆叠在夜间睡眠之上。柱子的主颜色由当天的作息状态动态决定

对于折线图，利用 `QPainterPath` 和 `QLinearGradient`（线性渐变），不仅绘制了连贯的睡眠评分折线，还在折线下方填充了一层淡蓝到透明的**渐变阴影区域面积图**。

自包含的数据载体

设计了一个专门的轻量级结构体 `SleepDayData`，类似于 `sleep_core` 的基类。它不仅封装了一天的核心数据（夜间睡眠、午睡分钟数、得分及熬夜状态标志），还利用内联函数提供了快捷数学转换接口，便于绘图函数调用：

```
double totalHours() const { return (nightsSleepMin + daysSleepMin) / 60.0; }
double nightHours() const { return nightsSleepMin / 60.0; }
```

key_monitor.h & key_monitor.cpp

键盘敲击注意力检测功能的上层界面。不关心当前程序运行在 Windows 还是 macOS，只负责通用的业务逻辑（如每 200ms 轮询一次计数、按分钟切分数据槽 `m_slots`、发送 Qt 信号等）。

重要工具：**条件编译（Conditional Compilation）**，用于和后面两个调用操作系统API的文件连接。

在 `key_monitor.cpp` 中，有以下代码：

```

#ifdef Q_OS_MACOS || defined(Q_OS_MAC)
extern "C" uint64_t macos_getSystemKeystrokeCount(); // 声明来自 macOS 文件的函数
#elif defined(Q_OS_WIN)
extern "C" uint64_t windows_getSystemKeystrokeCount(); // 声明来自 windows 文件的函数
#endif

static uint64_t getSystemKeystrokeCount() {
#ifdef Q_OS_MACOS ...
    return macos_getSystemKeystrokeCount();
#elif defined(Q_OS_WIN)
    return windows_getSystemKeystrokeCount();
#endif
}

```

- 编译期决定：当在 Windows 下编译程序时，CMake/Qt 编译器只会把 `key_monitor_win.cpp` 编译进去，并通过 `extern "C"` 让 `key_monitor.cpp` 成功调用到 `windows_getSystemKeystrokeCount()`；在 Mac 下编译时则同理调用 `macos_getSystemKeystrokeCount()`。
- 输入源统一：无论底层如何实现，最终都向跨平台的 `KeyMonitor` 提供一个自启动以来（或安装钩子以来）的**全局按键累计总数**（`uint64_t` 类型）。

key_monitor_win.cpp & key_monitor_mac.mm

键盘敲击注意力检测功能的底层操作系统适配层。调用了各自操作系统独有的系统级 API（Windows 的全局钩子 Hook，macOS 的 CoreGraphics 事件计数器），获取系统全局的按键总数。

`key_monitor.h` & `key_monitor.cpp` 这两个文件和 `key_monitor_win.cpp` & `key_monitor_mac.mm` 这两个文件，前者构成了软件的逻辑，定义了数据处理的模型；后者在底层实现上针对不同操作系统实现了适配数据处理模型，是典型的‘抽象与实现分离’的架构设计。

notification_manager.h & notification_manager.cpp

系统托盘与桌面气泡提醒管理器。起到了“程序后台常驻生命周期管理中心”与“非侵入式用户状态反馈引擎”的作用。

最小化到托盘管理

利用 Qt 的 `QSystemTrayIcon` 将程序常驻在操作系统的右下角（Windows）或右上角导航栏（MacOS）。

- 托盘图标的内部构建了一个 `QMenu`，绑定了“显示主界面”、“自定义设置”以及“彻底退出程序”的 `QAction`
- 在 `onTrayActivated` 槽函数中，通过捕获用户的点击行为（`Trigger` 或 `DoubleClick`），安全地恢复主窗口的状态

深夜熬夜行为的拦截

`checkLateNightCondition()` 函数是一个定时触发的“状态监测器”。

它通过一个每隔一定时间触发的 `QTimer`，在后台读取当前的系统时间（`QTime::currentTime()`）

当时间进入设定的熬夜区间1h以后时，主动通过 `QFile` 检索用户的本地作息 JSON 数据库，判断用户今天是否已经保存了入睡记录。

如果发现用户深夜未眠且未打卡，它会立刻警告气泡提示（`QSystemTrayIcon::Warning`）并播放警告音效，起到“健康守护”的干预效果。

格式化良好的接口

提供了一组格式化良好的公共接口，供软件的其他模块（如算分引擎、成就系统、主界面）调用，以桌面气泡的形式向用户传达信息

- `showSleepNotification(...)`：入睡成功记录反馈，并提示进入静默状态。
- `showWakeNotification(...)`：早安打卡反馈。
- `showAchievementNotification(...)`：当触发连续打卡 7 天或 30 天等成就时，由 `AchievementManager` 触发此接口向桌面推送勋章解锁通知。

弱耦合机制

`*NotificationManager` 类的构造函数里，接收一个 `QMainWindow*` 指针，但并没有将其硬编码强绑定，而是作为父对象（`QObject` 的单根继承体系）管理。它利用 信号与槽机制来异步通知主窗口。

```
signals:
    void requestShowWindow(); // 当用户双击托盘时激活该信号，由 MainWindow 监听并决定如何恢复窗口
```

这种方式可以实现主界面（UI层）与后台管理（逻辑层）的解耦，降低主窗口的负担。

系统音效调用

`NotificationManager` 类中设计了专门的系统音效触发函数（如 `playWarning()`、`playAchievement()`）。

在底层实现中，还可以通过 `QProcess::startDetached("afplay", ...)` 异步调用 macOS 系统的原生音频播放命令，实现了跨平台兼容和多进程机制。

自包含设计

另外一个很具有稳健性的设计是，使用了 `generateSleepIcon()` 静态辅助函数。利用 `QPainter` 在内存的 `QPixmap` 上通过纯代码直接绘制出了在托盘中显示的图标。不依赖外部图片的简单设计，让程序不会因为外部图片路径丢失而导致托盘图标不可见。

floating_widget.h & floating_widget.cpp

桌面悬浮球快捷打卡组件。独立于主窗口，始终停留在桌面最上层。基于 Qt 的常驻置顶与异形视窗机制。

鼠标事件驱动的拖曳

通过计算全局鼠标坐标与窗口左上角的相对偏移量（`m_dragOffset`），实现了在全屏范围内丝滑的拖拽移动体验。

无边框置顶窗口

在构造函数中，通过设置 Qt 的窗口标志（Window Flags），实现了悬浮球的窗口物理属性：

```
setWindowFlags(Qt::FramelessWindowHint | Qt::WindowStaysOnTopHint | Qt::Tool);
setAttribute(Qt::WA_TranslucentBackground); // 背景透明
setAttribute(Qt::WA_ShowWithoutActivating); // 启动时不抢占系统焦点
```


基于QPainter的矢量美术

悬浮球的视觉效果由 `paintEvent` 里的代码手工绘制。其内部利用了 `QLinearGradient`（双色环形/线性渐变阴影）渲染球体，并计算了像素点，绘制了数颗白色小星星和底部的“光泽反光层”。

manual.html

`manual.html` 是用户说明书，内容详见说明书本身。说明书通过 `resources.qrc` 打包进程序，启动时自动弹出，也可以通过主界面右上角的“？”按钮再次打开。

三、小组成员分工情况

王晨瑜：主要负责界面美化；完成了以下工作：

- 编写mac版本键盘检测功能
- 编写睡眠图表功能
- 参与api功能优化
- 项目录制
- 前端搭建与初步美化
- 任务栏设计
- 说明书初步撰写

董弈齐：主要负责问题探索与程序调试；完成了以下工作：

- 软件功能设计、软件灵感
- 在加入前端之前的基础C语言代码实现
- 编写成就功能
- 编写windows版本键盘检测功能
- 外观和图标优化
- windows版本和mac版本最终release
- 编写睡眠日记功能

曾梓航：主要负责Qt逻辑设计、搭建与调试；完成了以下工作：

- 时间逻辑设计与构建
- 日历构建
- api功能初步构建
- 自定义功能设计与构建
- 参与按钮与图表设计
- 参与多项功能优化、细节优化、注释撰写
- 参与项目录制
- 实验报告撰写
- 说明书后期撰写

四、项目总结与反思

4.1 项目完成情况总结

本次大作业中，我们完成了一个面向大学生群体的健康作息管理系统。相比最初只希望实现“记录入睡时间和起床时间”的简单设想，最终项目已经形成了比较完整的功能闭环：用户可以完成每日打卡、手动补录、通宵标记、睡眠评分、历史回顾、日历颜色标记、成就解锁、周报生成、图表查看、PDF 导出、键盘活跃度监测、系统托盘提醒和桌面悬浮球快捷打卡等操作。

在开发过程中，我们逐渐意识到，一个完整软件并不是把多个功能简单堆在一起，而是要让它们之间能够稳定联动。例如，用户点击“我醒了”之后，程序不仅要计算睡眠时长，还要保存 JSON 文件、刷新日历颜色、更新成就、生成当日简报，并为之后的周报和图表提供数据来源。这个过程让我们对“数据流”和“状态管理”有了更直观的理解。

从结果来看，项目基本达到了课程大作业的预期目标。它不仅能够运行，而且具备一定的实用性和完整性。尤其是作息日逻辑、跨平台键盘监测、系统托盘、图表可视化和 AI 风格周报等功能，使项目从一个普通的记录工具扩展成了一个更完整的桌面端健康管理软件。

4.2 开发过程中的收获

本项目最大的收获之一，是我们对 Qt 框架和图形界面程序开发有了更深入的理解。之前的程序大多是命令行输入输出，逻辑相对线性；而 Qt 程序需要处理按钮点击、窗口切换、信号槽连接、文件读写、网络请求、后台托盘、动态控件和绘图事件等内容。程序不再是“从上到下执行一次”，而是由用户行为不断触发不同事件。这种开发方式一开始并不容易适应，但也让我们真正体会到了事件驱动程序的特点。

第二个收获是模块化设计的重要性。项目早期的一些功能写得比较集中，后期功能变多之后，如果所有逻辑都堆在主窗口里，代码会很难维护。因此我们逐渐将功能拆分成不同模块：`sleep_core.h` 负责核心算法，`achievement_manager.h` 负责成就判断，`settings_dialog` 负责用户设置，`notification_manager` 负责托盘和提醒，`sleep_charts` 负责睡眠图表，`key_monitor` 负责键盘活跃度统计。这样的拆分让每个文件的职责更加清楚，也方便不同成员分别调试和完善。

第三个收获是对跨平台开发有了更具体的认识。Qt 本身提供了跨平台能力，但当功能涉及操作系统底层接口时，仍然需要分别适配。例如键盘活跃度监测在 Windows 上使用低级键盘钩子，在 macOS 上使用 CoreGraphics 相关接口，而上层则通过统一的 `KeyMonitor` 类进行处理。这让我们理解到，真正的跨平台开发并不只是“同一份代码到处编译”，还需要合理区分通用逻辑和平台相关逻辑。

第四个收获是对数据持久化和异常处理有了更多认识。程序需要把用户每天的数据保存下来，并且下次打开时能够正确读取。为此我们使用 JSON 文件保存每日作息数据和用户设置。开发过程中也遇到了文件不存在、字段缺失、临时数据、日期跨天、通宵记录等问题，这些都提醒我们，实际软件需要考虑很多非理想情况，不能只按照最顺利的输入流程来设计。

4.3 开发过程中的问题与不足

虽然项目最终完成度较高，但仍然存在一些不足。

首先，自动化测试还不够完善。我们虽然编写了测试数据生成脚本，用来快速生成多天睡眠记录，方便测试日历、图表和周报功能，但还没有建立系统化的单元测试。尤其是跨天睡眠、通宵、缺失数据、极端时间设置等边界情况，仍然主要依靠人工测试。后续如果继续完善，应该为核心算法和文件读写逻辑补充自动化测试。

其次，界面布局和视觉效果仍有提升空间。项目中加入了图标、悬浮球、图表和颜色标记等视觉设计，但整体仍然偏课程项目风格。如果要进一步做成正式软件，可以继续优化不同分辨率下的适配效果，减少部分硬编码坐标，并让界面风格更加统一。

第三，部分功能的分析维度还比较简单。例如睡眠评分主要依据睡眠时长、是否通宵和是否睡懒觉来判断，虽然符合课程项目需求，但还不能完全反映真实睡眠质量。键盘活跃度也只能代表电脑前的输入频率，不能准确判断用户是在学习、聊天还是娱乐。因此这些功能目前更适合作为辅助参考，而不能作为严格的健康判断依据。

第四，文档维护在前期不够同步。开发过程中，我们更关注功能实现，README、说明书和实验报告没有一开始就随着代码同步更新，导致后期需要集中补充项目结构、模块说明和使用说明。这也让我们意识到，文档不是最后才写的附属内容，而应该伴随开发过程持续维护。

4.4 关于开发工具使用的反思

在本次项目中，我们也借助了一些辅助工具来提高开发效率，例如用于查找报错原因、整理代码思路、润色说明文字和辅助检查部分逻辑。但总体来说，项目的需求设计、核心功能取舍、主要代码整合和最终调试仍然由小组成员完成。

这给我们的一个体会是：辅助工具可以提高效率，但不能代替对项目本身的理解。尤其是在本项目中，很多逻辑都和我们自己的需求密切相关，例如“作息日”如何归属、凌晨入睡应该算哪一天、通宵如何保存、日历颜色如何区分、用户设置如何影响评分等。如果只是机械地接受工具给出的代码，很容易出现表面能运行、实际逻辑不符合预期的问题。

因此，我们在开发中更倾向于先明确自己的设计思路，再使用工具辅助检查细节。这样既能提高调试和文档整理效率，也能保证我们对代码逻辑有足够掌握。通过这个过程，我们也认识到，合理使用工具的关键不是让它替我们完成整个项目，而是把它作为辅助，帮助我们更快发现问题、比较方案和整理表达。

4.5 最终反思

总体来看，这次项目让我们从“完成一个程序功能”进一步走向了“完成一个可运行的小型软件”。它包含界面、算法、文件、网络、图表、系统托盘、跨平台适配和用户文档等多个部分，开发过程中遇到的问题也比普通编程题更加综合。

这次经历让我们认识到，软件开发中最重要的不只是写出某一段代码，而是让整个系统的数据逻辑自治、功能流程闭环、模块职责清楚，并且在用户进行不同操作时都能保持稳定。一个按钮背后可能牵涉到时间判断、文件保存、界面刷新、成就判断和提示反馈；一个日期选择也可能需要处理有记录、无记录、临时记录、通宵记录等多种状态。这些细节让我们对工程化开发有了更实际的理解。

如果后续继续完善这个项目，我们希望重点改进三个方向：一是增加自动化测试，提高核心逻辑的稳定性；二是继续优化界面布局和交互体验；三是完善文档和项目管理流程，让代码、说明书和实验报告能够同步更新。

通过本次大作业，我们不仅复习和运用了 C++、Qt、JSON 文件处理、信号槽、绘图和跨平台编译等知识，也体会到了团队协作、模块划分、持续调试和文档整理的重要性。这些收获比单纯完成一个功能更加有价值。